DL-Practicals

Q) Boston House Price Prediction .

First understand the BIG IDEA
👉 Imagine this:
You want to **guess house price** 🏠
But instead of guessing randomly, you use data like:
- Number of rooms
- Area
- Location factors
- Tax

👉 This is exactly what your code is doing.
📌 So simple definition:
Machine Learning = Using past data to predict future values

🎯 **What type of problem is this?**
👉 This is called:
Regression Problem
📌 Why?
Because output is a **number (price)**, not category

Step-by-Step Explanation -

Step 1: Importing Libraries

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

👉 Think of libraries like tools:
- NumPy → math calculations
- Pandas → handle tables (like Excel)
- Matplotlib → draw graphs

Step 2: Load Dataset

```
from sklearn import datasets
boston = datasets.load_boston()
```

👉 You are loading a **ready-made dataset**
📌 It contains:
- 506 houses
- 13 features (details about houses)

Step 3: What are Features?

```
boston.feature_names
```

👉 Features = **inputs**
Example:
- CRIM → crime rate
- RM → number of rooms
- TAX → tax

👉 These help predict price

Step 4: Target (Output)

```
boston.target
```

👉 This is:
House price
📌 So:
- Input (X) → house details
- Output (Y) → price

Step 5: Create Table (DataFrame)

```
df = pd.DataFrame(boston.data)
df.columns = boston.feature_names
df['Price'] = boston.target
```

👉 Converts data into table form (like Excel)

Step 6: Understanding Data
✔ **df.head()**
👉 Shows first 5 rows
✔ **df.describe()**
👉 Gives summary:
- average
- min
- max

✔ **df.info()**
👉 Shows:
- data types
- missing values

📌 Important:
👉 No missing values → good dataset

Step 7: Split Data (VERY IMPORTANT)

```
train_test_split(x, y, test_size=0.2)
```

👉 You divide data into:
- **Training data (80%)** → model learns
- **Testing data (20%)** → model is checked

📌 Real-life example:
- Study → training
- Exam → testing

🔷 **Step 8: Linear Regression Concept**
👉 Model tries to draw a **best-fit line**

$y = mx + c$

y-intercept
x-intercept
👉 Meaning:
- y → predicted price
- x → inputs
- m → slope
- c → constant

👉 Model finds best values of m and c

🔷 **Step 9: Train Model**

```
lr = LinearRegression()
lr.fit(xtrain, ytrain)
```

👉 Model learns from training data
📌 Simple meaning:
👉 "Based on past data, find pattern"

🔷 **Step 10: Prediction**

```
y_pred = lr.predict(xtest)
```

👉 Model predicts price for new houses

🔷 **Step 11: Graph**

```
plt.scatter(ytest, y_pred)
```

👉 Shows:
- Actual price vs Predicted price

📌 If points are close → model is good

🔷 **Step 12: Error Calculation**
We check how wrong the model is

🔶 **1. Mean Squared Error (MSE)**

$$MSE = \frac{1}{n}\sum(y - \hat{y})^2$$

👉 Your result:
24.29
👉 Means:
👉 Bigger error is punished more
🔶 **2. Mean Absolute Error (MAE)**

$$MAE = \frac{1}{n}\sum|y - \hat{y}|$$

👉 Your result:

3.18
👉 Means:
👉 On average prediction is off by **3.18**

* In one line:
Data → **Train** → **Predict** → **Check Error**

Q) Sentiment Analysis Problem
📌 Simple meaning:
👉 Given a movie review → predict
👉 **Positive (1)** or **Negative (0)**

Real Life Example
👉 Review:
"This movie was amazing"
👉 Output:
👉 **Positive (1)**

👉 Review:
"Worst movie ever"
👉 Output:
👉 **Negative (0)**

📌 So:

Input	Output
Text (review)	Label (0 or 1)

👉 This is called:
*** Binary Classification Problem**

📦 2. Dataset used
From your file:

```
from keras.datasets import imdb
```

👉 This loads:
⭐ **IMDB Movie Review Dataset**
📌 Important points:
- 25,000 training reviews
- 25,000 testing reviews
- Reviews are already processed

3. Why numbers instead of text?
You saw this:

```
train_data[0]
```

👉 Output = numbers like:

```
[1, 14, 22, 16, 43, ...]
```

📌 Why??
👉 Because computer **cannot understand text directly**
👉 So words are converted into numbers

🔹 Example

Word	Number
"movie"	14
"good"	22

📌 This is called:
⭐ **Word Encoding**

4. Converting back to text
Your code:

```
decoded_review = ' '.join([...])
```

👉 Converts numbers → words
📌 So you can see actual review text again

5. Data Preprocessing (VERY IMPORTANT)
Your code:

```
def vectorize_sequences(...)
```

👉 This converts reviews into:
⭐ **Binary Vectors (0 and 1)**

- **So in a simple way -**

👉 Suppose vocabulary = 10,000 words
👉 Each review → array of size 10,000
👉 If word present → 1
👉 If not → 0

🧠 Example
Review: "good movie"
Vector:

```
[0,0,1,0,1,0,0,0,...]
```

📌 This is called:
⭐ **Bag of Words Representation**

6. Train and Test Data
Already given:
- `xtrain,ytrain`
- `xtest,ytest`

👉 Model learns from training
👉 Tested on testing

7. Neural Network Model
Your code:

```
model = models.Sequential()
```

👉 This means:
⭐ **Feedforward Neural Network**

*** Structure:**

```
Dense(16, activation='relu')
Dense(16, activation='relu')
Dense(1, activation='sigmoid')
```

🔶 Layer 1 & 2
👉 16 neurons
👉 Activation = ReLU
📌 Meaning:
👉 Helps model learn patterns

🔶 Final Layer
👉 1 neuron
👉 Activation = Sigmoid
📌 Output:
👉 Value between 0 and 1

👉 If > 0.5 → Positive
👉 If < 0.5 → Negative

📉 8. Activation Functions
🔹 ReLU

Q) Sentiment Analysis Problem
📌 Simple meaning:
👉 Given a movie review → predict
👉 **Positive** (1) or **Negative** (0)

Real Life Example
👉 Review:
"This movie was amazing"
👉 Output:
👉 **Positive (1)**

👉 Review:
"Worst movie ever"
👉 Output:
👉 **Negative (0)**

📌 So:

Input	Output
Text (review)	Label (0 or 1)

👉 This is called:
*** Binary Classification Problem**

2. Dataset used

From your file:

```
from keras.datasets import imdb
```

👉 This loads:
⭐ **IMDB Movie Review Dataset**
📌 Important points:
- 25,000 training reviews
- 25,000 testing reviews
- Reviews are already processed

3. Why numbers instead of text?

You saw this:

```
train_data[0]
```

👉 Output = numbers like:

```
[1, 14, 22, 16, 43, ...]
```

📌 Why??
👉 Because computer **cannot understand text directly**
👉 So words are converted into numbers

🔷 **Example**

Word	Number
“movie”	14
“good”	22

📌 This is called:
⭐ **Word Encoding**

4. Converting back to text

Your code:

```
decoded_review = ' '.join([...])
```

👉 Converts numbers → words
📌 So you can see actual review text again

5. Data Preprocessing (VERY IMPORTANT)

Your code:

```
def vectorize_sequences(...)
```

👉 This converts reviews into:
⭐ **Binary Vectors (0 and 1)**

🔷 **Simple idea**
👉 Suppose vocabulary = 10,000 words
👉 Each review → array of size 10,000
👉 If word present → 1
👉 If not → 0

🧠 **Example**
Review: “good movie”
Vector:

```
[0,0,1,0,1,0,0,0,...]
```

📌 This is called:
⭐ **Bag of Words Representation**

🔀 6. Train and Test Data

Already given:
- `xtrain`, `ytrain`
- `xtest`, `ytest`

👉 Model learns from training
👉 Tested on testing

🤖 7. Neural Network Model

Your code:

```
model = models.Sequential()
```

👉 This means:
⭐ **Feedforward Neural Network**

🔷 **Structure:**

```
Dense(16, activation='relu')
Dense(16, activation='relu')
Dense(1, activation='sigmoid')
```

🔶 **Layer 1 & 2**
👉 16 neurons
👉 Activation = ReLU
📌 Meaning:
👉 Helps model learn patterns

🔶 **Final Layer**
👉 1 neuron
👉 Activation = Sigmoid
📌 Output:
👉 Value between 0 and 1

👉 If > 0.5 → Positive
👉 If < 0.5 → Negative

📉 8. Activation Functions

🔷 **ReLU**

$$f(x) = \max(0, x)$$

👉 Removes negative values
👉 Makes training faster

🔷 **Sigmoid**

$$f(x) = \frac{1}{1+e^{-x}}$$

👉 Converts output into probability (0 to 1)

⚙️ 9. Model Compilation

```
model.compile(...)
```

🔶 **Optimizer: RMSprop**
👉 Helps update weights
👉 Improves learning

🔶 **Loss Function**

```
binary_crossentropy
```

👉 Used for **binary classification**
📌 Measures error between:
- actual value
- predicted value

🔶 **Metric**

```
binary_accuracy
```

👉 Measures:
👉 How many predictions are correct

10. Validation Split

```
xval = xtrain[:10000]
```

👉 Data divided into:

- Training
- Validation

📌 Why?
👉 To check model performance during training

11. Training the Model

```
model.fit(...)
```

🔷 **Epochs = 20**
👉 Model trains 20 times on data

🔷 **Batch size = 512**
👉 Data processed in chunks

📌 During training you saw:
- loss ↓ (good)
- accuracy ↑ (good)

12. Output Graph

You plotted:
- Training loss
- Validation loss

👉 Helps detect:
⭐ **Overfitting**

*** Overfitting meaning**
👉 Model remembers training data too much
👉 Performs worse on new data

- **Your full pipeline:**

Text → **Numbers** → **Vectors** → **Neural Network** → **Prediction (Positive/Negative)**

Q) Recognize handwritten digits (0–9) using Deep Neural Network

1. What is the problem?

This practical solves a **multiclass classification problem using a Deep Neural Network (DNN)**.
From your file
Problem statement:
Given an image of a handwritten digit, the model must predict which digit it is (0 to 9).

Simple understanding
- Input: Image of a digit (for example, image of “5”)
- Output: Predicted digit (5)

So the system learns to **recognize handwritten numbers**.

2. Type of problem

This is:
Multiclass Classification
Meaning:
- Output is not just 2 classes (like yes/no)
- Output can be one of many classes (0,1,2,…9)

3. Dataset used

The dataset used is **MNIST dataset**
It contains:
- 60,000 training images
- 10,000 testing images
- Each image is of size **28 × 28 pixels**

How image is stored
Each image is stored as a matrix:
- 28 rows × 28 columns
- Each value represents pixel brightness (0 to 255)

4. Data Preprocessing

Before giving data to the model, some important steps are done.

(1) Flattening

```
x_train.reshape(60000, 784)
```

Explanation:
- Each image (28 × 28) is converted into a single row of 784 values
- Because neural networks accept **1D input**

(2) Normalization

```
x_train /= 255
```

Explanation:
- Pixel values are between 0 and 255
- Converted to range 0 to 1

Reason:
- Makes training faster and stable

(3) One-Hot Encoding

```
y_train = np.eye(num_classes)[y_train]
```

Explanation:
Convert label into vector form:
Example:

Digit	Converted form
3	[0,0,0,1,0,0,0,0,0,0]

Reason:
- Required for multiclass classification

5. Neural Network Model

The model is created using a **Sequential Neural Network**.
Structure:
- Input layer: 784 neurons
- Hidden layer 1: 512 neurons
- Hidden layer 2: 512 neurons
- Output layer: 10 neurons

Activation Functions
ReLU (used in hidden layers)

$$f(x) = \max(0, x)$$

- Converts negative values to zero
- Helps model learn faster

Softmax (used in output layer)
- Converts output into probabilities
- Output values sum to 1

Example:
[0.01, 0.02, 0.85, …]
Highest value = predicted class

6. Dropout Layer

```
Dropout(0.2)
```

Explanation:
- Randomly removes 20% neurons during training
- Prevents overfitting

7. Model Compilation

```
model.compile(...)
```

Important components:
Loss Function:
categorical_crossentropy
- Used for multiclass classification
- Measures error between actual and predicted output

Optimizer:
RMSprop
- Updates weights to reduce error

Metric:
Accuracy
- Measures how many predictions are correct

8. Training the Model

```
model.fit(...)
```

Important terms:
- Epochs = 20
  → Model sees data 20 times
- Batch size = 128
  → Data processed in small groups

9. Model Evaluation

```
model.evaluate(...)
```

Output:
- Loss value
- Accuracy value

Your result:
Accuracy ≈ 98.5%
Meaning:
The model predicts correctly most of the time.

10. Final Flow of the program

Complete process:
Image → Flatten → Normalize → Neural Network → Prediction (digit 0–9)

11. Key concepts summary

- Multiclass Classification: More than 2 output classes
- Neural Network: Model with layers of neurons
- ReLU: Activation for hidden layers
- Softmax: Output probability function
- Dropout: Prevents overfitting
- Normalization: Scale data
- One-hot encoding: Convert labels to vector

*** This experiment performs handwritten digit recognition using a deep neural network and classifies images into one of ten classes using softmax activation.**

Q) Deep Learning based Classification (using Neural Network)

This practical is about:
Deep Learning based Classification (using Neural Network)
👉 In simple words:
The model learns from data and **predicts a category/class**.

Simple understanding

- Input → Data (can be image / text / numbers depending on your code)
- Output → Predicted class (category)

Type of problem

This is a **Classification Problem**
Depending on your file, it can be:
- Binary Classification → 2 outputs (0 or 1)
- Multiclass Classification → more than 2 outputs

2. Main Idea of the Practical

The goal is:
Train a Neural Network so that it can learn patterns from data and make predictions

3. Important Concepts -

1. Dataset

- Collection of data used for training
- Divided into:
  - Training data → used to learn
  - Testing data → used to check performance

2. Features and Labels

- Features (X) → input data
- Labels (Y) → correct output

Example:
- Input: Image of “5”
- Output: 5

3. Neural Network

A neural network is a model made of layers of neurons.
Structure:
- Input layer → takes data
- Hidden layers → learn patterns
- Output layer → gives result

4. Activation Function

Used to introduce learning ability.
ReLU (commonly used)

$$f(x) = \max(0, x)$$

- Converts negative values to zero
- Helps model learn faster

5. Output Activation

- Sigmoid → for binary classification
- Softmax → for multiclass classification

6. Loss Function

Measures error between:
- Actual output
- Predicted output

Examples:
- Binary crossentropy
- Categorical crossentropy

7. Optimizer

Updates model weights to reduce error
Example:
- RMSprop
- Adam

8. Training

```
model.fit(...)
```

Important terms:
- Epoch → number of times model sees data
- Batch size → number of samples processed at once

9. Evaluation

```
model.evaluate(...)
```

Gives:
- Loss
- Accuracy

10. Accuracy

- Shows how correct the model is

Example:
Accuracy = 95% → model is mostly correct

4. Full Flow of the Program

Data → Preprocessing → Neural Network → Training → Prediction → Evaluation

5. What you are learning from this practical

- How to build a neural network
- How to train it
- How to evaluate it
- How to classify data

This experiment uses a deep neural network to learn patterns from data and perform classification by predicting the correct class for given input.

Q) Image Classification using Convolutional Neural Network (CNN)

👉 Task:
Given an image (clothes), predict its class
Example: T-shirt, shoe, bag, etc.
👉 Dataset: **Fashion MNIST**

2. Type of Problem

Multiclass Classification
- Input → Image
- Output → One class out of 10

3. Dataset Concept

Fashion MNIST contains:
- 60,000 training images
- 10,000 testing images
- Image size: **28 × 28 pixels**

Image Representation

Each image = matrix
- 28 rows × 28 columns
- Each value = pixel intensity (0 to 255)

4. Data Preprocessing

(1) Normalization

```
x_train = x_train / 255.0
```

Explanation:
- Convert pixel values from 0–255 → 0–1
- Helps faster learning

(2) Reshaping

```
x_train.reshape(-1, 28, 28, 1)
```

Explanation:
- Add channel dimension

- Required for CNN

5. Convolutional Neural Network (CNN)

Definition

CNN is a neural network specially designed for **image processing**.

👉 It automatically learns:
- edges
- shapes
- patterns

6. Important CNN Layers

(1) Convolution Layer (Conv2D)

```
Conv2D(32, (3,3), activation='relu')
```

Explanation:
- Applies filters on image
- Extracts features

Example:
- Detect edges
- Detect shapes

(2) Activation Function (ReLU)

Explanation:
- Converts negative values to 0
- Helps model learn faster

(3) Pooling Layer (MaxPooling)

```
MaxPooling2D((2,2))
```

Explanation:
- Reduces image size
- Keeps important features

(4) Dropout

```
Dropout(0.25)
```

Explanation:
- Randomly removes neurons
- Prevents overfitting

(5) Flatten Layer

```
Flatten()
```

Explanation:
- Converts 2D data → 1D vector
- Required before Dense layer

(6) Dense Layer

```
Dense(128)
```

Explanation:
- Performs final learning
- Combines features

(7) Output Layer

```
Dense(10)
```

Explanation:
- 10 neurons → 10 classes
- Each neuron represents one class

7. Output Function

Softmax
- Converts output into probabilities
- Sum = 1

Example:

[0.1, 0.2, 0.6, …]

👉 Highest value = predicted class

8. Model Compilation

Loss Function

```
sparse_categorical_crossentropy
```

Explanation:
- Used for multiclass classification
- When labels are numbers (0–9)

Optimizer

```
adam
```

Explanation:
- Updates weights
- Improves learning

Metric

```
accuracy
```

Explanation:
- Measures correctness

9. Training Concepts

Epoch
- Number of times model sees full dataset

Batch Size
- Number of samples processed at once

10. Model Evaluation

```
model.evaluate(...)
```

Gives:
- Loss
- Accuracy

👉 Your result ≈ 91%

11. Overfitting

Meaning:

Model learns too much from training data

→ performs poorly on new data

Solution:
- Dropout
- More data
- Regularization

12. Full Flow

Image → Normalize → CNN Layers → Feature Extraction → Classification → Output